From Tool Use to Autonomy: Understanding the Rise of Agentic LLMs

Renu Rawal[1], Ariga Sri Naga Charani[2], Kunja Anusri[3], Dr.Kapil Kumar[4]
[1]2023KUCP1002@iiitkota.ac.in, [2]2023KUCP1143@iiitkota.ac.in, [3]2023KUCP1141@iiitkota.ac.in,
[4]kapil.maths@iiitkota.ac.in
[1234] Department of Computer Science and Engineering
Indian Institute of Information Technology, Kota, Rajasthan 325003, India

***Abstract*—Large language models (LLMs) are evolving from passive text generators into agentic systems capable of reasoning, planning, memory use, tool interaction, and self-correction. This survey synthesizes the neural foundations and architectural principles that enable agency in modern LLMs. We analyze transformer mechanisms for contextual reasoning, implicit planning, and heuristic computation, and introduce an agentic architecture stack integrating reasoning loops, hierarchical memory, verification modules, and tool-use interfaces. We review advances in structured reasoning, self-improvement, multi-agent coordination, and inference-time optimization, alongside case studies of frontier systems such as DeepAgent, BabyDragon Hatchling, BrainRot, SPICE, WebRL, and SWE-Agent. By unifying neural insights, architectural designs, and real-world implementations, this survey charts how LLMs progress toward adaptive, goal-directed autonomous intelligence**

Index Terms— Agentic Large Language Models, Autonomous Reasoning, Memory-Augmented Models, Tool Use and Function Calling, Hierarchical Memory Systems, Retrieval-Augmented Generation, Cognitive Architectures, Local Graph Models, Planning and Control, Multi-Step Reasoning, Distributed LLM Systems.

I. INTRODUCTION

Large language models have progressed from passive text generators to systems capable of structured reasoning, tool-mediated action, and adaptation to dynamic tasks. Early foundation models such as GPT-3 demonstrated broad generalization from an autoregressive training objective, yet they lacked mechanisms for goal decomposition, multi-step evaluation, external interaction, and persistent context maintenance. These limitations motivated a shift toward architectures that embed LLMs within cognitive processes such as planning, self-critique, memory-guided adaptation, and environmental feedback, forming what are now referred to as agentic LLMs.

A. From Scaling Laws to Agentic Cognition

Scaling laws initially drove progress in language modeling, but diminishing returns emerged on tasks involving symbolic reasoning and long-horizon decision making by 2023–2024 [2]. As a result, research increasingly focused on inference-time computation. Structured reasoning and self-improvement methods including Chain-of-Thought prompting [2], Tree-of-Thought search [6], multi-pass self reflection [4][5], and intrinsic reward optimization as explored in DeepSeek-R1 [13] ,demonstrated that improved reasoning quality depends not only on parameter count but on computation that unfolds during inference.

This shift reflects a broader evolution in system design. Instead of operating in a single forward pass, agentic LLMs perform guided reasoning over multiple steps while interacting with tools, retrieving external information, and revising intermediate conclusions. Early external-agent frameworks such as AutoGPT and AutoGen illustrated the potential of this paradigm, but their reliance on rigid planning loops exposed scalability challenges. Newer systems integrate these abilities within the model itself. DeepAgent, for example, supports internal planning, tool selection, tool invocation, and iterative verification directly in the model's reasoning cycle. Figure 1 provides a high-level illustration of this integrated workflow.

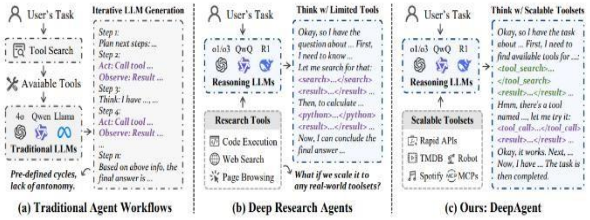

Fig. 1. Agentic LLM workflow. Planning, tool execution, and iterative verification represented using DeepAgent-style architecture [12].

B. Positioning of This Survey and Key Contributions

Existing surveys, including Plaat et al., primarily categorize systems based on user-visible behaviors such as reasoning and acting. They provide limited coverage of the mechanisms that enable these behaviors: transformer-level interpretability, control structures for reasoning, memory governance for adaptation, and architectures supporting collective multi-agent cognition. Frontier developments such as SPICE for symbolic inference [18], WebRL for webgrounded action [19], DeepAgent for scalable reasoning orchestration [12], and studies of degradation dynamics such as Brain Rot [17] remain inadequately synthesized in current literature.

This survey addresses these gaps by offering:

- a neural-level examination of transformer properties relevant to agentic cognition, including induction heads and representational control mechanisms [29][30]
- a unified architectural model capturing planning, self critique, memory, tool integration, and verification as coordinate components
- coverage of advanced inference-time optimization frameworks, including structured search and reflective improvement
- analysis of memory systems that support long-term adaptation and state persistence

• discussion of multi-agent workflows and organizational structures for collaborative problem solving

• synthesis of industrial implementations and frontier research prototyping real-world autonomy

By integrating these perspectives, the survey aims to provide a compact and comprehensive reference for understanding the design principles and emerging capabilities of agentic LLMs.

II. Neural Foundations of Agentic Cognition

A. Transformer Substrates Enabling Agency

The shift from static language modeling toward autonomous cognition originates in the inductive biases of the transformer architecture. Multi-head self-attention enables selective information routing over long contexts, allowing an LLM to construct internal task states that behave similarly to short-term working memory [1]. This core operation is defined as:

$$\text{Attention}(Q, K, V) = \text{Softmax}(QK^T / \sqrt{d_k})V$$

giving the model the ability to bind and manipulate relational structure inside text sequences [29][30]. Mechanistic studies reveal specialized "induction heads" that detect and continue latent patterns, functioning as algorithmic primitives for sequential reasoning [30]. Neural circuit analyses further show modular internal pathways that support abstraction and constraint satisfaction, indicating emergent symbolic-like operations inside purely neural systems [29].

These insights suggest that transformers contain the foundational substrates required for deliberation. Figure 2 illustrates how attention heads connect to form reasoning supportive circuits, based on transformer interpretability results from Nadarajah et al. [29].However, these circuits operate only during a single forward pass and dissolve immediately afterward, preventing any persistent goal tracking or state maintenance beyond the context window [32]. Thus, while the neural substrate permits reasoning, it does not enable adaptive, ongoing cognitive control.

In other words, transformer reasoning is fundamentally stateless. Each forward pass must reconstruct any relevant intermediate concepts from the current token sequence, without the ability to directly update internal memory structures. As soon as the sequence changes, the previously "constructed" reasoning circuit disappears, and a new one is formed from scratch. This makes planning, strategy revision, task-monitoring, or delayed-reward evaluation extremely difficult without external scaffolding.

To compensate for this statelessness, recent agentic LLM systems increasingly rely on external scaffolds such as memory modules, tool-use pipelines, and planning controllers that operate across multiple forward passes. These components supply the persistent state and iterative feedback loops that transformers inherently lack, effectively transforming momentary neural computations into extended cognitive processes capable of goal maintenance, self-correction, and multi-step deliberation.

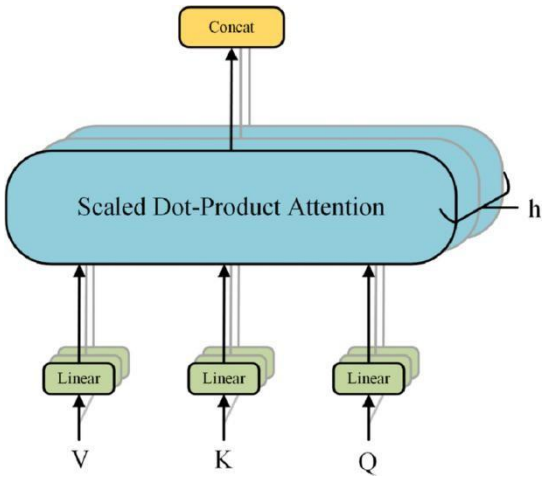

Fig.2.Emergent Transformer Circuits for Reasoning Based on: Nadarajah et al., 2024 [29]

B. Why Internal Mechanisms Alone Are Insufficient

Transformers simulate deliberation but lack persistent feedback. They cannot evaluate their own correctness or adjust behaviour dynamically, which is essential for autonomy. This leads to instability when required to self correct or learn from unfolding experience [32]. Over time, self-generated supervision can degrade internal representations, a phenomenon identified as "Brain Rot" [17]. Moreover, transformers are not embodied. Their reasoning has no inherent connection to external tools, actuators, or environmental signals, preventing grounded decision-making [8].

To transition from passive prediction to autonomous agency, transformers must be integrated with additional components: planning for goal decomposition [3], reflection and verification loops for quality control [4], intrinsic reinforcement signals during inference to stabilize reasoning policies [13], and long-term memory systems to preserve learned context [14][15]. Tool-use interfaces are also essential to convert reasoning into action within digital or physical environments [8][9]. Only with these extensions can models execute iterative cognition that follows a Goal → Action → Evaluation → Self-Improvement trajectory, the defining characteristic of agentic intelligence.

III. Research Questions

Agentic large language models are studied from many angles: neural mechanisms, architectural design, inferencetime algorithms, memory, tools, and multi-agent ecosystems. To keep the review focused and systematic, we frame the survey around a small set of guiding research questions.

RQ1. Neural foundations

What internal mechanisms of transformer based LLMs support agent like cognition, in particular planning, working memory, and structured reasoning?

This question focuses on the model substrate itself. It seeks to understand how attention patterns, induction heads, emergent circuits, and other interpretability findings relate to the capacity for agentic behaviour, and where these mechanisms are insufficient for autonomy without additional structure.

RQ2. Architectural organization

How are LLMs embedded within broader architectures that combine planning, memory, tool interfaces, and verification to form agentic systems?

Here the emphasis is on the system level. The question and control loops are arranged around a core LLM to support goal decomposition asks how planners, critics, memory modules, tool connectors,, long horizon execution, and safety checks, and which design patterns recur across different agentic frameworks.

RQ3. Inference time reasoning and self improvement

Which inference time strategies and self improvement mechanisms enhance or degrade the reasoning performance of agentic LLMs?

This question covers methods such as chain of thought, tree and graph structured reasoning, self critique and multi pass decoding, reinforcement style intrinsic rewards for reasoning, and self generated data loops. It also includes negative results, such as degradation under low quality or self produced data, that constrain how such methods can be used safely.

RQ4. Systems, applications, and failure modes

What capabilities, limitations, and failure modes emerge in real world agentic LLM systems that combine neural foundations, architecture, tools, and memory at scale?

The final question connects the previous three to practice. It looks at deployed or high fidelity prototypes such as DeepAgent, SWE Agent, WebRL, SPICE, BrainRot, BabyDragon Hatchling, and AI Scientist, and asks what they reveal about scalable cognitive architectures, safety risks, and open research problems.

The rest of the paper is organised to address these questions in a structured way. Section 2 focuses on RQ1 by examining neural and interpretability findings. Sections 3 to 6 address RQ2 and RQ3 through the lens of architecture, reasoning control, tool use, and memory. Section 7 answers RQ4 by synthesising findings from frontier case studies. Section 8 distils the resulting research gaps and future directions.

IV. Methodology

This work follows a systematic literature review process adapted to the rapidly evolving nature of agentic LLM research. The goal of the methodology is not only to collect influential works, but also to make the selection process transparent and reproducible.

A. Scope and Time Frame

The review targets research on large language models that exhibit agentic behaviour. This includes works on:

- neural mechanisms that support agent like cognition,
- architectural frameworks that embed LLMs in control loops,
- inference time reasoning and self improvement,
- memory augmented and tool using LLM systems,
- multi agent and organisational level frameworks.

Because most agentic LLM research has emerged recently, the primary time window considered is *January 2020 to April 2025*. Earlier foundational works are included selectively when they are essential for context, for example early transformer papers or classical planning references cited by later agentic systems.

B.Data Sources

We queried a combination of established digital libraries and preprint servers:

- IEEE Xplore
- ACM Digital Library
- SpringerLink
- Elsevier ScienceDirect
- Scopus
- arXiv (cs.AI, cs.CL, cs.LG, cs.MA categories)
- Selected project repositories and technical reports from major research laboratories and companies working on agentic LLMs.

The inclusion of arXiv and technical reports is important in this domain, since many frontier systems are first released as preprints or system cards before formal publication.

C. Search Strategy

To capture the broad and evolving terminology associated with agentic LLMs, search keywords were organized into three conceptual categories. The first category, Model Type, includes terms such as large language model, LLM, foundation model, and transformer. The second category focuses on Agency and Behaviour, represented by terms like agent, agentic, autonomous, planning, tool use, tool calling, memory-augmented, reasoning, and self-improvement. The third category reflects the Application or System Context, using keywords such as web agent, software engineering agent, cognitive architecture, multi-agent system, agent framework, and LLM operating system. Together, these categories provide comprehensive coverage of terminology relevant to the study of agentic LLMs.

Representative query templates included:

- (“large language model” OR “LLM”) AND (“agent” OR “agentic” OR “autonomous”)
- (“transformer”) AND (“planning” OR “tool use” OR “API calling”)
- (“LLM agent”) AND (“memory” OR “long-term memory”)
- (“multi-agent”) AND (“LLM” OR “agentic organisation”)

Database filters were applied for Computer Science and Artificial Intelligence where supported.

(D) Inclusion and Exclusion Criteria

The study selection procedure was conducted through a systematic, multi-layered review workflow to ensure methodological rigour and consistency. Initially, all records retrieved from database searches, citation tracking, and manual scanning of reference lists were consolidated into a reference management system. At this stage, automated and manual identification of duplicate entries was performed to avoid redundancy. Following deduplication, a **title- and abstract-level screening** was carried out, during which studies that were clearly unrelated to the scope of agentic LLM architectures, lacked a technical component, or focused on conventional non-LLM reinforcement learning or robotics were excluded.

Papers that passed the initial screening were then subjected to a detailed **full-text evaluation**, where the predefined inclusion and exclusion criteria were applied systematically. This included assessing whether the study demonstrated agentic behaviour, described planning or tool-use mechanisms, provided meaningful insights into memory or control loops, or offered sufficient technical depth for methodological interpretation. Studies that appeared ambiguous, unclear, or partially relevant were re-examined through iterative discussion and cross-validation to minimise selection bias.

Finally, only those studies that met at least one inclusion criterion and provided adequate methodological transparency were retained for qualitative synthesis. The overall process ensured that the final corpus of literature was both technically relevant and representative of current advances in agentic LLM systems, while excluding non-technical, duplicative, or out-of-scope works.

D. Study Selection Procedure

The selection process followed a structured pipeline similar to PRISMA guidelines.

Step 1: Initial Retrieval

The initial search produced 312 records across all data sources. After removing duplicates based on title and DOI, 247 unique records remained.

Step 2: Title and Abstract Screening

Titles and abstracts were screened using the inclusion and exclusion criteria. Studies clearly unrelated to agentic LLM behaviour were removed. After this stage, 68 papers were retained.

Step 3: Full-Text Review

Full texts were assessed for:

- •Technical depth regarding agentic components
- •Clear methodology and architectural description
- • Relevance to at least one research question

The final corpus included 37 primary research papers and 9 secondary or survey papers, yielding a total of 46 studies. A PRISMA-style diagram can be added to visually represent this process.

V . Agentic Architecture Stack

The development of agentic Large Language Models marks a major shift from simple prompt-response systems toward AI that can operate with greater independence. Early systems such as ReAct demonstrated that combining reasoning with action execution allows models to interact with the world rather than only generate text [3]. This idea paved the way for systems like AutoGPT and Auto-Agents, where an LLM can set goals, execute plans, call external tools, and refine its own output [9], [28]. As research progressed, agentic systems started evolving from singleagent automation into multi-agent ecosystems capable of collaboration, negotiation, and shared reasoning [10], [24]..

A. Cognitive Core, Memory, Planning, and Action Execution

The foundation of an agentic system lies in how reasoning, planning, and memory are integrated. Models like DeepAgent demonstrate how scalable tool-use frameworks allow an LLM to interact with increasingly complex environments [12], while multi-step planning frameworks such as SPICE introduce symbolic reasoning to improve task decomposition and constraint handling [18].

Memory is a critical differentiator between agenti systems and standard language models. Instead of treating every user request as a new conversation, the agent retains task history, execution logs, and contextual knowledge similar to how SWE-Agent maintains state across software engineering tasks [21]. With memory available, the agent can adapt to user preferences, resume interrupted tasks, and incrementally improve performance over time.

Planning and execution modules bridge the gap between reasoning and action. Using reinforcement learning approaches such as WebRL, the system can learn through trial-and-error how to interact with digital environments more effectively [19].

As automation scales, execution may involve interacting with APIs, running structured workflows, or coordinating actions among multiple specialized subagents.

B. Thought → Action → Verification Loop

A defining characteristic of modern agentic systems is the iterative cycle of reflection and correction. Rather than executing a plan blindly, the agent observes the results of its actions and determines whether adjustments are needed. Iterative optimization approaches, such as those explored in black-box output refinement [31], demonstrate the effectiveness of self-evaluation in improving model performance.

Multi-agent systems like AutoGen extend this principle by distributing roles: one agent may act as a planner, another as an executor, and another as a critic or verifier [10]. By separating reasoning tasks, the system gains resilience, parallelism, and the ability to challenge its own output— much like human team-based problem solving.

C. Failure Patterns

Despite rapid progress, agentic systems are still far from flawless. Common failures include hallucinated tool calls, unstable planning loops, and increasing divergence from the intended task—particularly during long-horizon execution [33], [34]. Safety risks emerge when agents operate without

reliable verification, especially in real-world software, financial, or web environments.

Several studies emphasize the need for structured safeguards, grounded reasoning strategies, and better monitoring mechanisms to avoid unintended or unsafe behavior [24], [33]. As agentic systems become more autonomous and embedded in organizational workflows, addressing these vulnerabilities becomes not only a technical challenge but a requirement for responsible deployment.

VI. Inference-Time Intelligence and Self-Improvement

A. Structured Reasoning: CoT, ToT, GoT

Inference-time intelligence starts with how an LLM structures its own intermediate thoughts. Chain-of-Thought prompting (CoT) [2] asks the model to reveal a sequence of intermediate reasoning steps rather than jumping directly to an answer. This transforms the model's internal computation into an observable "reasoning trace," which functions as an external working memory and leads to large gains on arithmetic and symbolic reasoning benchmarks. However, the structure of CoT is strictly linear: the model commits to a single path, and any early mistake can propagate to the final answer.

Tree of Thoughts (ToT) [6] generalizes CoT by treating coherent reasoning segments as "thoughts" and organizing them into a search tree that can branch, backtrack, and prune. At each step, the system expands multiple candidate thoughts, scores them with either heuristics or an auxiliary model, and chooses which branches to extend. This converts inference into deliberate search over a structured space of possible reasoning trajectories. Graph of Thoughts (GoT) [7] extends this idea further by representing thoughts and their relations as an arbitrary graph, where multiple reasoning paths can diverge and later merge into shared sub-solutions. This graph structure is better suited for elaborate tasks where partial solutions must be reused across branches.

To make this concrete for the reader, you can use a sideby-side schematic:

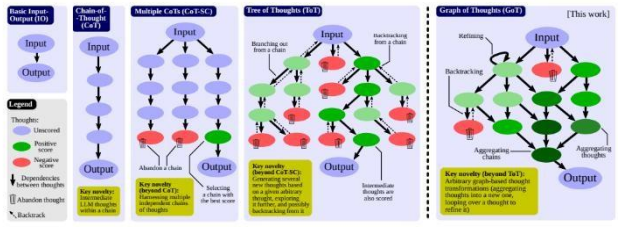

Fig.3. Structural comparison of CoT, ToT, and GoT[2].

From an agentic-cognition perspective, these methods are not merely prompt tricks. They define what the model is allowed to consider during inference and how it navigates that space. CoT corresponds to a single possible world; ToT explores a branching set of futures; GoT explores a network of partially overlapping futures. Section 4.1 is therefore about the geometry of thought space at inference time.

B. Self-Critique and Multi-Pass Reasoning

Structured thought alone does not guarantee good reasoning. The model must also be able to evaluate and revise its own trajectories. Reflexion [4] introduces a mechanism where, after executing a trajectory (e.g., solving a task in a text-based environment), the agent produces a separate "reflection" describing what went wrong and how it should change strategy next time. This reflection is stored in memory and conditions subsequent episodes, effectively turning natural language critiques into a form of verbal reinforcement.

Verbalized Sampling [5] builds on this idea by sampling multiple distinct rationales, then selecting or reweighting them based on internal quality signals. Conceptually, many of these methods can be written as optimizing over a critic function on thought trajectories. If we denote a thought trajectory by (\tau) and a scalar critique or score by (C(\tau)), then the system aims to favor trajectories with higher scores. In abstract form, this selection process can be expressed as a common approach in agentic LLMs is to weight each reasoning trajectory τusing a critic $C(\tau)$, often expressed through a Boltzmann-style rule:

$$p(\tau) \propto \exp(\lambda C(\tau)),$$

where λ controls how strongly the critique influences selection. This ensures that trajectories with better self-assessed quality are exponentially more likely to be kept or expanded.

In practice, the critic $C(\tau)$may be:

- a rule-based verifier (e.g., test-case pass rate)
- a learned reward model
- *an LLM acting as a meta-judge*.

Black-box iterative improvement methods [31] wrap this into an outer optimization loop, where the base LLM remains fixed but prompts, intermediate steps, or tool choices are tuned to maximize some task metric over repeated trials.

C. Inference-Time Reinforcement and Self-Generated Data Loops

Some recent systems go beyond selection and critique, adding explicit reward signals to shape how models reason. DeepSeek-R1 [13] is a prominent example: it trains models to prefer deep, logically consistent reasoning traces by assigning higher rewards to high-quality chains of thought and optimizing the model with reinforcement learning. Before writing the formal objective, it is useful to frame it conceptually:

The goal is to find model parameters that maximize the expected reward of sampled reasoning trajectories, where reward encodes final correctness and internal consistency. Formally, this can be written as

Other approaches keep the base model fixed but use similar reward ideas in an outer loop. Black-box optimization [31] may repeatedly query the model with different prompts or tool configurations, observe task rewards (e.g., success or failure), and gradually adapt the orchestration strategy. In this case, the agent around the model learns a better inference policy, even though the model weights do not change.

Self-generated data loops sit on top of these mechanisms. Systems such as AI Scientist [20], Deep Research [25], and SWE-Agent [21] automatically create subproblems,

intermediate documents, or synthetic examples during operation and then reuse them to guide further reasoning or to fine-tune specialized models. This can significantly amplify capabilities but also introduces a key reliability problem: when the generated data drifts away from ground truth, the system risks reinforcing its own biases and hallucinations.

The "Brain Rot" study [17] provides an empirical warning. It shows that repeated exposure to low-quality, engagement-optimized, or self-generated content can measurably degrade reasoning: chains of thought become shorter, reliance on shallow heuristics grows, and performance on challenging tasks declines. In terms of your outline, this is Section 4.4: Self-Generated Data Loops and Reliability. The same loops that enable autonomous improvement can also cause cognitive decay if feedback and data quality are not carefully controlled.

Architectures like MemGPT [14] and A-MEM [15] respond by structuring memory as filtered episodes rather than raw logs, so that only high-signal, relevant experiences are available for retrieval. Tool-grounded systems such as SPICE [18] and WebRL [19] further constrain self-generated content by tying it to executable plans or real web interactions, making it harder for the model to drift into selfconsistent but false narratives.

The message of Section C is therefore two-sided:

- properly designed inference-time reinforcement and data loops can stabilize and enhance reasoning,
- but poorly constrained loops can destabilize cognition, a risk that must be addressed explicitly in later safety sections.

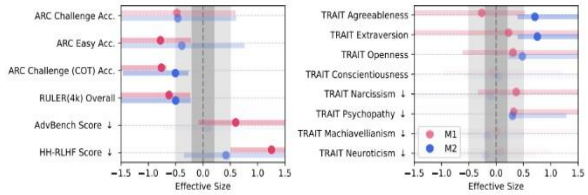

Fig. 4. Reasoning degradation under low-quality data exposure[17].

VII .Tool Use and Digital Interaction in Agentic LLMs

Agentic LLMs rely heavily on external tools to overcome the inherent limitations of static parametric knowledge, bounded context windows, and restricted numerical or symbolic reasoning capacity. Tool use allows models to access realtime information, extend their computational abilities, execute software workflows, and interact with digital environments such as APIs or web interfaces. The shift from pure text generation to action-conditioned intelligence was first formalized by ReAct, which blended chain-of-thought reasoning with tool invocation to enable mixed reasoning– acting loops [3]. Since then, a large ecosystem of toolenabled frameworks has emerged, each expanding the autonomy and capabilities of LLM agents.

Tool use is generally motivated by two fundamental drivers. The first is knowledge extension, where tools such as retrieval systems, search APIs, and databases compensate for LLMs' limited factual freshness. The second driver is functional extension, where models delegate tasks they cannot perform internally—such as code execution, mathematical computation, file manipulation, or interacting with GUIs. This dual motivation positions tools as cognitive prosthetics, enabling LLMs to operate as general-purpose digital workers.

Early systems such as Toolformer demonstrated that LLMs can self-supervise their own tool-learning by identifying beneficial API calls during training [8]. This approach highlighted a crucial insight: LLMs not only use tools but can *decide when* tool invocation is beneficial. Subsequent work built on this foundation by integrating tool calls into structured control loops. ReAct introduced step-by-step reasoning traces that alternate between verbal thoughts and external actions, allowing the agent to ground its internal reasoning with evidence from the environment [3]. Reflexion further enhanced this process by enabling agents to reflect on previous failures and strategically revise their tool-use strategies through verbalized self-feedback [4]. Together, these systems established the modern paradigm of LLMs as tool-driven problem solvers.

A major breakthrough in practical tool-enabled autonomy came with systems such as AutoGPT and BabyAGI, which operationalized continuous task decomposition, planning, and execution using external tools [9], [11]. AutoGPT demonstrated that even a single LLM, when coupled with filesystem access, search tools, and execution modules, could autonomously perform multi-step tasks. However, the absence of reliable verifiers and the tendency for hallucinated tool calls often caused unstable behavior. AutoGen expanded this idea using multi-agent collaboration, where specialized agents communicate to invoke tools, evaluate results, and refine plans [10]. This architecture proved more stable because communication between agents provided additional self-correction signals.

More recent frameworks emphasize robust and safe tool use at scale. SWE-Agent showed that tool-enabled LLMs can perform complex software engineering tasks by interacting with an entire tool suite, including code runners, debuggers, version-control systems, and file explorers [21]. These systems treat tools not as isolated APIs but as part of a structured environment with constraints, dependencies, and side effects. DeepAgent extended this paradigm by introducing scalable toolsets and a reasoning core capable of integrating signals from hundreds of external tools in real time [12]. Such systems illustrate a shift toward operatingsystem-level tool orchestration for LLMs.

A parallel line of research focuses on enabling LLMs to interact with the graphical web. WebRL treated web navigation as an RL environment, allowing LLMs to learn interface manipulation and DOM-based tool usage through reinforcement learning [19]. Google DeepMind's agentic Gemini and OpenAI's Deep Research framework similarly incorporate browsing actions as first-class tools, enabling agents to gather evidence, verify claims, and perform iterative refinement [25], [26]. These systems demonstrate that tool use is evolving from fixed API calls into adaptive digital

interaction, where the agent dynamically selects between search, navigation, form filling, and multi-step web workflows.

Despite these advancements, tool use in LLM agents remains error-prone. Tool hallucination where agents fabricate APIs or misuse existing actions remains one of the most persistent challenges [33]. Even small mismatches between the expected and actual tool outputs can lead to cascading failures, especially in autonomous loops with weak verification. Some frameworks attempt to mitigate this through execution monitors, critics, or dual-channel reasoning, but reliable tool-grounded behavior is still an open research problem. Models also struggle with tool overuse, where unnecessary API calls slow execution, or tool underuse, where the agent fails to invoke a needed tool due to miscalibrated confidence.

In summary, tool use defines the practical boundary of what modern agentic LLMs can accomplish. From simple API extensions to complex multi-agent systems with web interaction and software-level control, tool-enabled autonomy transforms LLMs from text generators into general-purpose digital actors. However, stability, safety, and robust grounding remain open challenges that limit fully autonomous deployment. Addressing these issues is central to future progress in agentic LLM architectures.

VIII. Memory and Adaptation in Agentic LLM Systems

Memory plays a defining role in transforming Large Language Models (LLMs) from passive text generators into capable autonomous agents. Traditional LLMs operate in a stateless manner, meaning each prompt is processed independently without persistent context. However, agentic LLM systems require the ability to remember goals, track progress across tasks, reflect on past decisions, and adapt based on environmental feedback. This shift makes memory not just a technical addition, but a foundational requirement for reasoning-driven autonomy.

Early frameworks such as ReAct demonstrated how even short-term reasoning traces could improve decision-making by enabling agents to alternate between thought and action [3]. Later systems including AutoGPT and BabyAGI expanded on this idea by incorporating persistent memory via external vector storage, allowing agents to accumulate experience and refine task breakdown strategies over long-running sessions [9], [11]. While effective, these early memory systems lacked structure, often resulting in redundant, noisy, or uncurated memory growth.

To address these issues, structured memory architectures emerged. One of the most influential contributions is

MemGPT, which introduced a hierarchical memory model inspired by cognitive science, distinguishing between shortterm working memory and long-term persistent storage [14]. This architecture allows the agent to dynamically "page in" relevant memory while "paging out" unused information, enabling continuous reasoning despite limited context length.

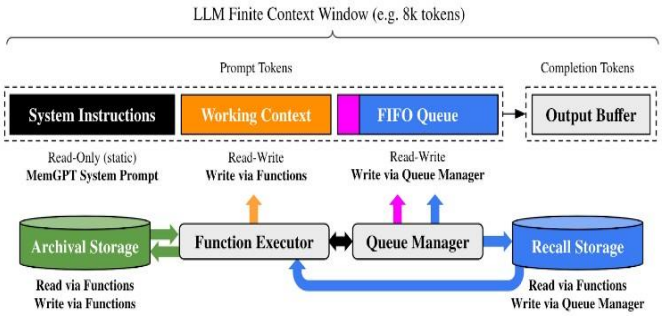

Fig.5.Hierarchical Memory-Augmented LLM (MemGPT)

Building upon MemGPT, recent work such as A-MEM introduces prioritization, relevance scoring, and forgetting mechanisms, ensuring that only meaningful experiences persist while outdated or irrelevant information fades over time [15]. This marks an important transition from memory storage to memory management where agents curate knowledge rather than simply accumulating it.

Memory is increasingly becoming collaborative as well. In multi-agent ecosystems such as AutoGen and CAMEL, memory is shared or negotiated between agents to support teamwork, role-based specialization, and emergent communication patterns [10], [22]. At greater scale, simulation environments like OASIS demonstrate how memory-driven interactions may give rise to emergent norms, social dynamics, and collective reasoning behaviors [23].

Adaptation, however, introduces risk. For example, studies on degradation of self-generated data show that continuous learning from an agent's own outputs may gradually distort model accuracy or factual grounding [17]. Similar concerns appear in emergent self-modifying systems such as Anthropic's Baby Dragon Hatchling, where adaptive behavior becomes unpredictable as agents learn from their own trace outputs [16]. These findings highlight the delicate trade-off between learning autonomy and system stability.

Across these developments, the trajectory of agentic memory systems can be summarized as follows:

- Stateless prediction → Persistent episodic storage → Hierarchical memory → Self-maintaining adaptive memory → Shared cognitive memory across multi-agent systems.

In essence, memory and adaptation transform LLMs into evolving cognitive systems capable of reflection, growth, and long-term autonomy. While breakthroughs demonstrate clear progress, they also introduce open questions regarding safety, interpretability, scalability, and alignment. As agentic systems continue to mature, responsible design of memorybased adaptation will be critical to ensuring agents remain useful, reliable, and controlled.

IX. Industry Systems & Case Studies

A.System Objectives & Innovation

It is set of deterministic bash and edit tools, constraining the agent's action space to verifiable file system and command-line operations. The Live-SWE-Agent variant further advances Runtime Self-Evolution of its own agent code.

BabyDragon Hatchling (BDH) [16]: BDH tackles the problem of continuous, in-context self-evolution and the inherent opacity of the Transformer architecture. It is necessary for autonomous cognition by seeking a biologically-plausible mechanism for working memory and reasoning that is scale-free and natively interpretable. BDH advances Inference-time Synaptic Plasticity by using a graphbased, Hebbian-like learning mechanism within its recurrent update cycle, allowing memory and concept association to be updated locally on its "synaptic" edges without full gradientbased retraining.

LLM BrainRot [17]: This work empirically addresses the critical, system-level problem of representational drift and cognitive degradation in models continuously exposed to low-quality, 'junk' data. Its necessity lies in ensuring the longterm reliability and safety alignment of continuouslydeployed LLMs. It pushes forward the agentic capability of Cognitive Health Monitoring and Resilience, by quantifying the persistent decline in reasoning and long-context understanding post-junk exposure, even after post-hoc mitigation efforts.

DeepAgent [12]: DeepAgent targets the core challenge of long-horizon, complex, multi-modal tasks that overwhelm the context window of 'shallow' ReAct or CoT agents. It is necessary for scaling autonomous systems to solve real-world problems requiring days or weeks of computation. It advances Hierarchical Planning and State Management by decoupling the orchestrator from specialized sub-agents and externalizing working memory into persistent storage (e.g., file systems, vector DBs), allowing for context isolation and unbounded task length

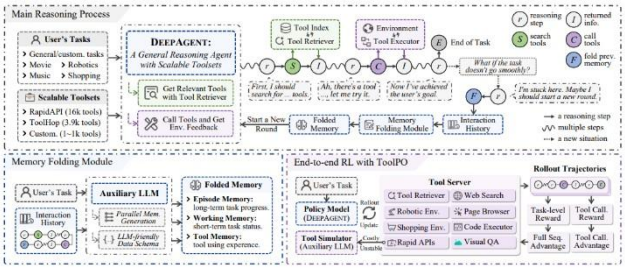

Fig.6. DeepAgent: End-to-End Tool Use with Memory Folding

SPICE [18]: SPICE addresses the fragility of neural planning and the challenge of grounding LLM actions in formal logic. It is necessary for autonomous cognition in safety-critical or mathematically rigorous environments where verifiable, symbolic execution is mandatory. It advances Neuro-Symbolic Planning and Verification by integrating LLMs as semantic parsers/proposers within a framework that uses classical, logic-based solvers (e.g., theorem provers, formal language engines) to generate and validate computational graphs.

WebRL [19]: WebRL solves the problem of training proficient web-navigation agents with sparse, binary

feedback signals from complex web environments. It is necessary for achieving general-purpose, high-success-rate web-based task completion. It advances Online Reinforcement Learning with Self-Evolving Curriculum by using an Outcome-supervised Reward Model (ORM) and a self-evolving curriculum that autonomously generates new, challenging tasks from previous failure cases, mitigating data scarcity and policy drift.

AI Scientist (Kosmos) [20]: This system addresses the goal of full-cycle autonomous scientific discovery from hypothesis generation to experimental design and result synthesis. It is necessary to accelerate the pace of scientific breakthroughs by removing human-in-the-loop bottlenecks. It advances Iterative, Evidence-Grounded Research through an autonomous cycle of literature search, data analysis, and world-model update, synthesizing complex findings into scientific reports traceable to source evidence.

SWE-Agent [21]: SWE-Agent focuses on automating the highly structured domain of Software Engineering, specifically fixing bugs in existing codebases. It is necessary for automated software maintenance and development at scale. It pushes forward Grounded Tool Use and Code Editing by integrating the LLM with a minimal set of deterministic bash and edit tools, constraining the agent's action space to verifiable file system and command-line operations. The Live-SWE-Agent variant further advances Runtime Self-Evolution of its own agent code.

B.Architectural Mechanisms

Table 1. System Architecture Breakdown: Planning, Action Execution, and Self-Correction in Agentic LLMs

Architectural Mechanism	System Observation	Thought Plannig	Action Execution	Verification / SelfCorrection
DeepAgent [12]	Failed tool execution	Orchestrator revises explicit plan	Subagnt redeleation	Plan update ensures correct execution
SPICE [18]	LLM-proposed plan	Transform plan into formal languae	Solver executs the plan	Deterministic solver output ensures logical soundness
WebRL [19]	Web environment state	Policy proposes next action	Agent executes web action	Outcome supervised reward modelevalus success
SWE-Agent [21]	Context window + tool output	Decide bash/edit command	Execute command on codebase	External test execution validates results

C.Performance Insights & Emergent Behaviors

Agentic LLM systems exhibit several notable empirical breakthroughs, particularly in reliability, representation stability, and adaptive behavior. BabyDragon Hatchling [16] achieves GPT-2-class performance while exhibiting **monosemantic activation patterns**, where sparse positive neuron activations map cleanly to distinct humaninterpretable concepts. This transparency emerges from its Hebbian plasticity layer, where internal "synapses" strengthen during concept-specific reasoning, offering an intriguing biologically inspired mechanism for rapid internal state adaptation.

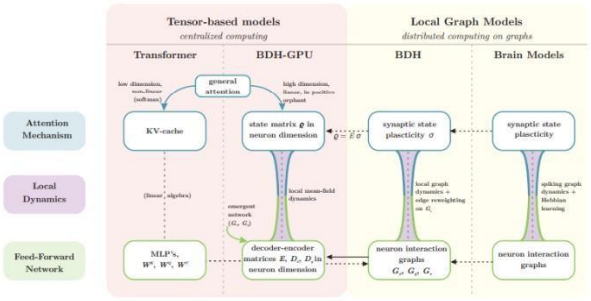

Fig. 7. Comparison of Tensor-Based and Local Graph Models

LLM BrainRot [17] provides a contrasting insight by revealing how continuous exposure to degraded or lowquality data induces persistent **representational drift** inside transformers. This drift manifests in failure modes such as thought-skipping prematurely truncating multi-step reasoning and measurable degradation in benchmarks such as ARC-CoT, where accuracy drops from 74.9% to 57.2%. Notably, the damage remains only partially recoverable even after extensive instruction tuning, underscoring the long-term risks posed by uncontrolled self-generated supervision.

WebRL [19] demonstrates robust performance on the WebArena-Lite benchmark with a 70.1% success rate, substantially surpassing imitation-learning baselines. Its most significant emergent behavior is a **self-evolving curriculum**, in which the agent autonomously formulates progressively harder but solvable tasks. This mechanism leads to continuous policy refinement and improved resilience in noisy, dynamic web environments.

SWE-Agent [21] reaches state-of-the-art results in automated bug fixing, achieving a 75.4% solve rate on SWE-bench Verified. Its reliability stems from a deliberately constrained action space (bash and edit operations only), which tightens verifiability. The more advanced Live-SWE-Agent variant shows an emergent form of **online architectural selfadaptation**, dynamically rewriting its own scaffolding code during execution to update tool APIs or adjust control-flow logic when encountering unseen error classes. This behavior represents one of the clearest real-world demonstrations of runtime structural self-modification in an LLM-based agent.

D.Research Synthesis

Table 2. Research Synthesis of Agentic LLM System Patterns and Their Key Mechanisms

Pattern	Representative Systems	Key Mechanism	Purpose / Advantage
Hierarchical Decomposition & External State	DeepAgent [12], WebRL [19]	Recursive task decomposition; sub-agents with persistent external memory (file system, vector DB)	Overcomes context window limitations; enables longhorizon, complex task handling
Hybrid Reasoning & Verification	SPICE [18], AI Scientist [20]	LLM proposals integrated with external deterministic computation (formal solvers, empirical data)	Mitigates unreliability of LLM only reasoning; ensures verifiable outputs; shifts LLM to semantic parser/proposer role

- Hierarchical Decomposition and External State: Systems like DeepAgent address complexity by recursively decomposing tasks and isolating context within specialized sub-agents, relying on persistent, external memory (e.g., file systems, vector databases) [14], [35]. This pattern overcomes the context window bottleneck of monolithic LLMs.
- Hybrid Reasoning and Verification: SPICE and AI Scientist integrate the LLM's generative capacity with external, deterministic, and verifiable computation. This mitigates the inherent unreliability of LLM-only reasoning by grounding it in formal logic (SPICE) or empirical data (AI Scientist), shifting the LLM's role from a sole reasoner to a semantic parser/proposer [18].

The most critical common bottlenecks are:

- Catastrophic Forgetting/Degradation: Empirically proven by LLM BrainRot, the assumption of static or continually improving LLM knowledge is false, mandating robust, proactive data curation and continual cognitive health checks [30].
- Scalable Self-Improvement: While WebRL and SWE-Agent demonstrate on-policy and runtime self-adaptation, a general, gradient-free, and truly unbounded architectural self-evolution remains a frontier challenge, though BDH offers a compelling in-context biological analogue.

Evolution Toward Scalable Cognitive Architectures These prototypes inform the evolution toward scalable cognitive architectures by defining the necessary submodules of a truly autonomous agent operating in the open world:

- System 1/System 2 Duality: The SPICE approach confirms the need for a Neuro-Symbolic split, where fast, intuitive LLM proposal (System 1) is systematically filtered and executed by a slower, verifiable System 2 (solvers/provers).
- Deep Memory Hierarchy: DeepAgent established the necessity of a memory hierarchy: ephemeral context → short-term external working state → long-term vector knowledge base [15], [34
- Endogenous Learning and Resilience: The findings from BDH and LLM BrainRot underscore that scalable systems must incorporate endogenous mechanisms for maintenance: in-context fast weight updates for working memory and explicit rot mitigation protocols for persistent knowledge.

The future of dependable, agentic AI systems is not a larger monolithic model but a Modular, Hybrid Cognitive Architecture that orchestrates specialized LLM reasoners with deterministic tools, verifiers, and hierarchical state management [24], [25].

X. Future Directions: Scaling Reasoning Quality, Sustaining Cognitive Reliability, and Structuring Collective Agency

The trajectory of recent agentic LLM developments suggests that future research will be shaped not only by improvements in parameter count and training data, but also by how models organize and evaluate their own reasoning processes during deployment. The next generation of systems will require coherent integration of inference-time control, stability preservation, and multi-agent coordination to operate reliably in open-ended environments.

A.Scalable Reasoning Through Inference-Time Control

Studies indicate that substantial improvements in reasoning emerge when a model's decision process is deliberately structured. Instead of producing a single linear solution path, inference-time control mechanisms allow exploration of alternative hypotheses, evaluation of intermediate evidence, and principled rejection of failureprone branches. These behaviors expose latent competencies already embedded in pretrained models. For example, Chainof-Thought prompting enables explicit multi-step justifications and improves PaLM-540B performance on GSM8K. Expanding this to Tree-of-Thought search yields significant further gains, including improvements in symbolic task performance without modifying model parameters [2][6].

This trend is effectively summarized through empirical results showing that introducing structure into the reasoning pipeline leads to markedly improved accuracy on reasoning benchmarks.

Table 3. Performance improvements from structured inference.
Source: Data derive from [2],[6].

Method	Model	Benchmark	Standard Accuracy	Structured Accuracy
CoT prompting	PaLM-540B	GSM8K	17.9%	58.1%
ToT search	GPT-4	Game of 24	4%	74%

Future systems will likely formalize these strategies into dedicated control components that govern navigation through the solution space, determining when to branch, when to prune, and when to delegate computation to external tools. Thus, reasoning becomes an explicit computational process rather than an incidental by-product of language generation.

B. Stability of Long-Term Autonomous Cognition

As LLMs generate their own intermediate outcomes and persist knowledge across episodes, maintaining internal consistency becomes essential. Unfiltered self-generated content can reinforce local heuristics or erroneous associations, gradually distorting the model's ability to perform structured reasoning. The survival of cognitive integrity therefore depends on the design of principled memory governance, where new information is validated before storage, and access is mediated to prevent uncontrolled accumulation of misleading or corrupted internal traces.

In an agentic setting, stability must be treated as a primary system property rather than an after-deployment correction. The architecture must provide explicit mechanisms to monitor behavioral drift, verify the logical validity of evolving reasoning strategies, and ensure that autonomous improvement remains anchored in high-quality signal. This shift from episodic prompting to continuous cognition raises new research challenges concerning long-term evaluation, adaptive calibration, and fault detection within autonomous cognitive loops.

C.Collective Agency and Distributed Cognitive Organization

While a single model may struggle with extended planning, tool-dependence, or self-critique, distributing responsibilities among specialized agents has shown to yield more reliable and interpretable behavior. Collaborative frameworks allow planning, execution, and verification processes to be separated across agents, enabling mutual oversight and modular refinement. This supports robustness through redundancy, as well as more transparent attribution of responsibility within the reasoning process.

Future progress will likely align agentic LLM design with principles from organizational intelligence. Systems may increasingly resemble coordinated institutions equipped with shared memory, communication policies, and governance mechanisms that ensure internal

accountability. Accordingly, evaluation metrics will evolve from single-model correctness toward assessments of collective competence, safety, and adaptability across multi-role deployments.

XI. Conclusion

The evolution of large language models (LLMs) into fully agentic systems marks a turning point in AI: we are no longer dealing primarily with passive text generators, but with adaptive agents capable of reasoning, acting, memory-driven adaptation, and long-term goal pursuit. This survey has traced how this transition rests on two intertwined developments: (1) neural and algorithmic foundations enabling internal reasoning, and (2) architectural designs that integrate reasoning, memory, tool use, and verification into a coherent cognitive stack.

At the neural level, transformer architectures through mechanisms such as self-attention and emergent internal circuits provide the substrate for contextual reasoning, pattern recognition, and latent planning capabilities. Yet, as shown by interpretability and mechanistic studies, these internal operations remain ephemeral, dissolving after a single forward pass. This temporality limits persistence of goal-state or memory beyond the model's context window, making pure transformer-based reasoning insufficient for sustained, adaptive cognition.

To overcome these limitations, agentic architectures embed LLMs within broader control structures: planners, memory modules, tool interfaces, and verification loops. Systems like ReAct illustrate the power of interleaving reasoning (thought) with action enabling dynamic interaction with external tools and environments rather than static answer generation [3]. Similarly, structured inference-time reasoning strategies such as Chain-of-Thought prompting (CoT), Tree of Thoughts (ToT), and generalizations like Graph of Thoughts (GoT) allow LLMs to explore multiple reasoning trajectories, improving performance on complex tasks requiring planning or search [27], [33].

Memory and long-term adaptation emerge as critical differentiators: architectures such as memory-augmented LLMs allow agents to retain knowledge, track progress, and build on prior experiences. For example, mechanisms like those proposed in Think-in-Memory (TiM) demonstrate how LLMs can maintain and update memory over conversations enabling more consistent, coherent, and history-aware behavior over time [34]. These innovations transform LLMs from stateless responders to evolving agents with episodic and semantic memory akin to human cognition.

On the applied front, real-world agentic systems combining reasoning, memory, tool use, and multi-step workflows have begun to deliver tangible results, from toolgrounded web navigation to symbolic problem solving and autonomous decision making [9], [10]. These systems illustrate both the promise and the challenges of agentic LLMs: the potential for adaptive, long-horizon autonomy, and the need for robust safeguards, verification, and memory governance.

Looking ahead, the next generation of agentic systems will likely emphasize modular, hybrid cognitive architectures. These would combine neural reasoning with symbolic verification, structured memory hierarchies, tool orchestration, and multi-agent coordination. Achieving reliable, safe, and adaptable autonomy will require not just bigger models, but smarter architectural design, principled control of inference-time behavior, and careful memory management ensuring that agents remain coherent, aligned, and resilient even as they learn and evolve.

In sum: agentic LLMs represent a convergence of neural foundation, cognitive architecture, and system-level engineering. The path from generative language model to autonomous cognitive agent is no longer speculative it is actively being charted [3], [9], [10], [27], [33], [34]. Continued progress will depend on rigorous integration of reasoning, memory, tool use, and verification, guided by both empirical results and principled design.

ACKNOWLEDGMENT

We express our sincere gratitude to our instructor, Dr. Kapil Kumar, for his never-ending support, guidance, and encouragement during our research concerning Progressive Web Applications. It is under his great expertise that this work has evolved, renu rawal , charani and Anushri for the collaboration and camaraderie that made this journey enjoyable and enriching. Thanks also to those who gave great feedback while writing this article. At last, but not least, we thank the reviewers for their thoughtful comments, which have considerably improved this manuscript.

REFERENCES

[1] T. Brown et al., "Language Models are Few-Shot Learners," NeurIPS, 2020.

[2] J. Wei et al., "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," NeurIPS, 2022.

[3] S. Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models," ICLR, 2023.

[4] R. Shinn, B. Labash, and M. Ghassemi, "Reflexion: Language Agents with arXiv:2303.11366, 2024. Language Verbal Self-Reflection,"

[5] M. Kim et al., "Verbalized Sampling for Self-Improving Large Models," arXiv:2502.05641, 2025. Stanford University,

[6] Y. Wu et al., "Tree of Thoughts: Deliberate Problem Solving with Large Language Models," arXiv:2305.10601, 2023.

[7] A. Kazemnejad et al., "Graph of Thoughts: Solving Problems with Large Language Models in Structured Ways," arXiv:2308.09668, 2023.

[8] J. Schick et al., "Toolformer: Language Models Can Teach Themselves to Use Tools," arXiv:2302.04761, 2023.

[9] Significant Gravitas, "AutoGPT: An Autonomous GPT-4 Experiment," GitHub, 2023.

[10] H. Zhang et al., "AutoGen: Enabling Next-Gen LLM Applications via arXiv:2308.08155, 2023. Multi-Agent Collaboration,"

[11] T. Richards, "BabyAGI: Autonomous Task-Driven AI Agent," GitHub, 2023.

[12] T. Li et al., "DeepAgent: A General Reasoning Agent with Scalable Toolsets," arXiv:2510.21618, 2025.

[13] DeepSeek-AI, "DeepSeek-R1: Intrinsic Reinforcement Reasoning in LLMs," Technical Report, 2025.

[14] Y. Xu et al., "MemGPT: Towards LLMs with Long-Term Memory," arXiv:2310.08560, 2023.

[15] S. Chen et al., "A-MEM: Agentic Memory for Autonomous LLM Agents," arXiv:2504.05218, 2025.

[16] Anthropic Research, "Baby Dragon Hatchling: Emergent Self-Evolving Abilities in Large Language Models," arXiv:2408.06725, 2024.

[17] A. Arora et al., "LLMs Can Get Brain Rot: Self Generated Data Degradation in Large Language Models," arXiv:2405.12065, 2024. Cognitive

[18] S. Hao et al., "SPICE: Symbolic Planning and Inference for Environments," Stanford University, arXiv:2403.04195, 2024.

[19] X. Huang et al., "WebRL: Learning to Use the Web with Reinforcement Learning," arXiv:2401.09363, 2024. Google DeepMind,

[20] A. Dragan et al., "AI Scientist: Autonomous System for Scientific Discovery," arXiv:2408.09839, 2024.

[21] A. Qian et al., "SWE-Agent: Agentic Large Language Models for Software Engineering," arXiv:2407.00279, 2024.

[22] T. Li et al., "CAMEL: Communicative Agents for Mind Exploration," arXiv:2303.17760, 2023.

[23] B. Park et al., "OASIS: Open-Agent Social Interaction Simulator," arXiv:2402.06772, 2024.

[24] Microsoft Research, "The Era of Agentic Organizations," arXiv:2502.12032, 2025.Organizations," arXiv:2502.12032, 2025.

[25] OpenAI, "Deep Research: Multi-Agent Iterative Reasoning Framework," Technical System Card, 2025.

[26] Google DeepMind, "Gemini 2.0 & AutoGPT2 Architecture," Technical Report, 2025.

[27] Anthropic, "Claude 3.7 Reasoner: Advanced Chain-of Thought Model," System Paper, 2025.

[28] Meta AI Research, "Auto-Agents: Towards Autonomous Multi-Agent Architectures," Technical Report, 2024.

[29] A. Nadarajah et al., "Neuroscience of Transformers: Emergent Internal Circuits for Reasoning," Nature Machine Intelligence, 2024.

[30] N. Olsson et al., "Induction Heads and Mechanistic Interpretability in Transformers," NeurIPS, 2022.

[31] H. Zuo et al., "Black-Box Optimization of LLM Outputs with Iterative Improvement," arXiv:2409.11234, 2024.

[32] J. Kirk et al., "Long-Context Failures and Retrieval Biases inLLMs," arXiv:2311.12223, 2023.

[33] K. Borg et al., "Safety Challenges in Autonomous LLM Agents," arXiv:2406.00812, 2024.

[34] J. Plaat et al., "Agentic Large Language Models: A Survey," arXiv:2503.23037, 2025.

[35] R. Zhang et al., "AgentOS: Towards Operating Systems for Autonomous LLMs," arXiv:2409.15012, 2024.